PART B

1. A compiler design team needs a tool to convert arithmetic expressions from infix notation to postfix notation for efficient code generation. The expression contains alphanumeric operands, operators (+, -, *, /, %, ^), and brackets: (,), {, }, [,]. Develop a C program that supports the following:
   a. Read an infix expression.
   b. Validate the expression for correct bracket nesting and syntax.
   c. Convert the valid infix expression to postfix notation.

   Note: An input expression is valid if:
   • Operators must have valid operands on both sides.
   • Open brackets must be closed by the same type of brackets.
   • Open brackets must be closed in the correct order.

   Every close bracket has a corresponding open bracket of the same type.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
// Priority of stack operators
int F(char symbol)
{
   switch(symbol)
   {
      case '+':
      case '-': return 2;
      case '*':
      case '/':
      case '%': return 4;
      case '^': return 5;
      case '(':
      case '{':
      case '[': return 0;

      case '#': return -1;

      default: return 8;
   }
}
// Priority of incoming operators
int G(char symbol)
{
```

```
    switch(symbol)
    {
        case '+':
        case '-': return 1;
        case '*':
        case '/':
        case '%': return 3;
        case '^': return 6;
        case '(':
        case '{':
        case '[': return 9;
        case ')':
        case '}':
        case ']': return 0;

        default: return 7;
    }
}

// Check matching brackets
int match(char a, char b)
{
    return (a=='(' && b==')') ||
           (a=='{' && b=='}') ||
           (a=='[' && b==']');
}
// Validate expression and brackets
int valid(char s[])
{
    char st[MAX];
    int top=-1, expect=1;
    for(int i=0; s[i]!='\0'; i++)
    {
        char c=s[i];
        if(isspace((unsigned char)c))
            continue;
        if(isalnum((unsigned char)c))
        {
            if(!expect) return 0;
            expect=0;
        }
        else if(c=='(' || c=='{' || c=='[')
        {
            if(!expect) return 0;
            st[++top]=c;
        }
        else if(c==')' || c=='}' || c==']')
        {
```

```
            if(expect || top==-1 || !match(st[top],c))
                return 0;

            top--;
            expect=0;
        }
        else if(c=='+' || c=='-' || c=='*' ||
                c=='/' || c=='%' || c=='^')
        {
            if(expect) return 0;
            expect=1;
        }
        else
            return 0;
    }

    return top==-1 && !expect;
}
// Convert infix to postfix
void infix_postfix(char infix[], char postfix[])
{
    char s[MAX];
    int top=-1, i, j=0;
    char symbol;

    s[++top]='#';

    for(i=0; infix[i]!='\0'; i++)
    {
        symbol=infix[i];
        if(isspace((unsigned char)symbol))
            continue;

       if(isalnum((unsigned char)symbol))
        {
            postfix[j++]=symbol;
        }
        else if(symbol=='(' || symbol=='{' || symbol=='[')
        {
            s[++top]=symbol;
        }
        else if(symbol==')' || symbol=='}' || symbol==']')
        {
            while(top>=0 && s[top]!='(' &&
                  s[top]!='{' && s[top]!='[')
                postfix[j++]=s[top--];

            top--;
```

```
        }
        else
        {
            while(F(s[top]) > G(symbol))
                postfix[j++]=s[top--];

            if(F(s[top]) != G(symbol))
                s[++top]=symbol;
            else
                top--;
        }
    }

    while(s[top]!='#')
        postfix[j++]=s[top--];

    postfix[j]='\0';
}

// Main function
int main()
{
    char infix[MAX], postfix[MAX];

    printf("Enter infix expression: ");
    if(fgets(infix,MAX,stdin)==NULL)
        return 1;

    infix[strcspn(infix,"\n")]='\0';

    if(valid(infix))
    {
        infix_postfix(infix,postfix);
        printf("Valid expression\n");
        printf("Postfix expression: %s\n",postfix);
    }
    else
    {
        printf("Invalid expression\n");
    }

    return 0;
}
```